Techniques for the Generation of Arbitrary Three-Dimensional Shapes in Tile-Based Self-Assembly Systems

Florian-Lennert Adrian Lau
lau@itm.uni-luebeck.de
Institute of Telematics
Lübeck, Germany

Kristof Stahl
kristof.stahl@student.uni-luebeck.de
Institute of Telematics
Lübeck, Germany

Stefan Fischer
fischer@itm.uni-luebeck.de
Institute of Telematics
Lübeck, Germany

ABSTRACT

A big challenge in nanorobotics is the construction of nanoscale objects. DNA is a reliable, bio-compatible tool to reliably and constructively create objects at the nanoscale. A possible tool to build nano-sized structures are tile-based self-assembly systems on the basis of DNA.

It is challenging and time-consuming to efficiently design blueprints for the desired objects. This paper presents basic algorithms for the creation of tilesets for $n \times n \times n$-cubes in the aTAM model. Most research in mathematical self-assembly models focuses on two-dimensional constructs. Only few publications focus on three-dimensional DNA crystals [**?**]. Three-dimensional shapes are likely to be of more use in nanorobotics. We present three variations: hollow cubes, cube-grids and filled cubes.

The paper also presents a basic algorithm to create arbitrary, finite, connected, three-dimensional and predefined shapes at temperature 1, as well as ideas for more efficient algorithms. Among those are algorithms for spheres, ellipsoids, red blood cells and other promising designs. The algorithms and tilesets are tested/verified using a simulation engine influenced by the well-known tool ISU TAS. Others can use the simulator and the algorithms to quickly create sets of tiletypes for their desired nanostructures. A long learning process may thus be omitted.

CCS CONCEPTS

• **Theory of computation → Models of computation**; • **Computing methodologies → Model development and analysis**; • **Applied computing → Telecommunications**; • **Networks**;

KEYWORDS

tile-based self-assembly; computational complexity; aTAM; nanodevices

NANOCOM '18, Sep. 17-19, 2017, Reykjavik, Iceland

ACM ISBN 978-1-4503-4931-4/17/09…$15.00
https://doi.org/10.1145/3109453.3109458

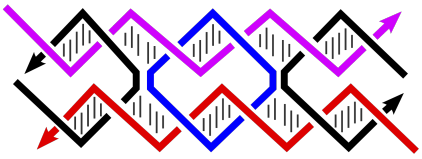


Figure 1: The basic two-dimensional molecule that enables the implementation of tile-based self-assembly systems [6].

ACM Reference Format:
Florian-Lennert Adrian Lau, Kristof Stahl, and Stefan Fischer. 2018. Techniques for the Generation of Arbitrary Three-Dimensional Shapes in Tile-Based Self-Assembly Systems. In *Proceedings of NANOCOM '18.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3109453.3109458

1 INTRODUCTION TO NANOROBOTICS

Sicknesses and injuries are among the oldest human ailments. Many generations of alchemists and scientists tried to find solutions to these problems. Nanotechnologies and especially nanomedicine is the next logical step in the long lasting search for a general cure.

In nanomedicine, nanorobots are often proposed for the treatment of diseases in countless scenarios. Some concepts for nanorobots are based on DNA as a building block [18]. In 2005 and 2006 Rothemund [16] and Seeman [19] presented several nanoscale two-dimensional objects that were assembled out of DNA. Since then, Akyldiz [1] proposed nanonetworks as a design principle for nanoscale computing/robotics. However, the actual implementation of nanorobots still remains unclear to this day.

In [12], a 3D DNA-nanomachine has been realized and tested in vivo. One of the basic components for the self-assembly of DNA-crystals are DX-molecules (see Figure 1). These molecules can be used for DNA computing because their open ends can interact with those of other molecules. Other, three-dimensional building blocks have also been realized. Few publications focus on three-dimensional DNA crystals [**?**]. Since three-dimensional structures have already been realized and due to a variety of applications, we assume that the discipline will progress in the near future.

To tackle the problem of creating nanorobots, we present an approach for the algorithmic construction of simple, three-dimensional objects based on tiles. Since there are no available tools to manipulate matter at the atomic scale, it appears advantageous to use self-organization for the assembly process. Our solution is based on the abstract Tile Assembly Model (aTAM) from Eric Winfree [22].

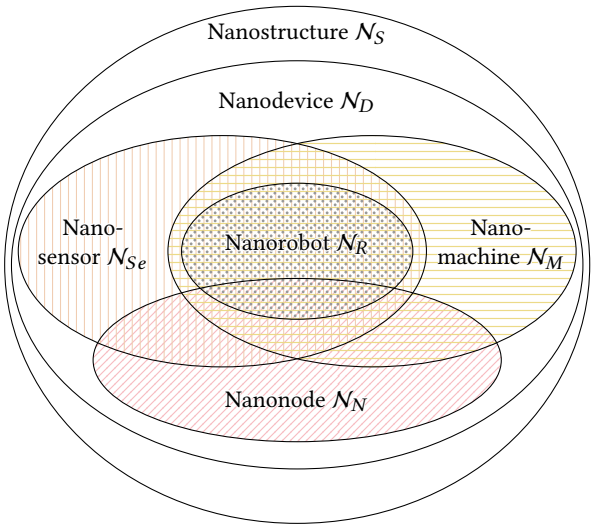


Figure 2: An overview of various nanostructures and their relationships.

The resulting DNA-based structures may be used to shelter medical payload [12] (with DNA "keys" to open the lid of a nanobox, e.g.) or serve as a "casing" to connect components like batteries and circuits. Many other practical scenarios are possible.

The rest of the paper is structured as follows. Section 2 explains the mathematical preliminaries for Tile Assembly Systems. Section 3 shows an algorithm for different types and sizes of cubes. Section 4 generalizes the basic algorithm to arbitrary three-dimensional shapes. Section 5 analyzes the algorithmic complexity of the algorithms and simulation tools and Section 6 summarizes the paper and explains open problems.

2 MODELLING PRELIMINARIES

Since there are numerous definitions for different nanodevices, we introduce a formal definition for the constructs the presented algorithms yield. In new areas like nanonetworking, words like "nanobot", "nanorobot", 'nanomachine" and "nanodevice" are often used synonymously. Every person has a different concept for the respective words. We also briefly define the required terminology to understand Tile Assembly Systems.

In [5] Büther et al. introduced a hierarchy for various. Figure 2 depicts all formally defined nanostructures.

The newly introduced term is *nanostructures*, denoting the most primitive type of nanoscale constructs. Nanostructures require no functional component other than a framework that constitutes its physical appearance. They may be described as a simple three-dimensional shape. Formally:

Definition 2.1. A *nanostructure* $\mathcal{N}_S = K_{\text{opt}}$ is a nanoscale, artificial construct, designed to fulfill a predefined function in an environment Γ. A nanostructure consists of 0 or more optional components $K_{\text{opt}} \subseteq \{A, C, I, L, M, P, S, T\}$.

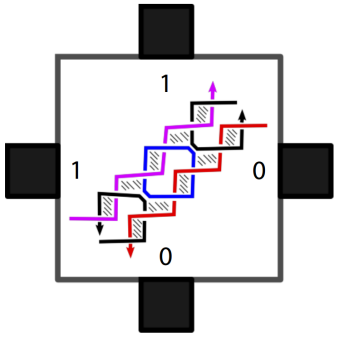


Figure 3: An overlay of the DX-molecule and a mathematical tile.

For the sake of this paper, we focus on nanostructures in their most basic form, i.e., without any optional components. The possible components are *Actuation* (A), *Computation* (C), *Information processing* (I), *Locomotion* (L), *Memory* (M), *Power Supply* (P), *Sensor* (S) and *Time* (T). The Environment Γ may be the human body. The function of nanostructures might be the connection of the aforementioned components.

In contrast, *nanodevices* require at least a power supply *P*. *Nanomachines* additionally have an actuatory *A* component to manipulate the environment. *Nanosensors* require a sensory component *S* instead of an actuator. *Nanorobots* are the most specific kind of nanomachine. They require all previously mentioned components and locomotion *L*, as well as information processing *I* and memory *M*. *Nanonodes* require at least a power supply *P* and the ability to communicate *C*.

The basic components for self-assembly systems are *tiles* [14]. The idea was originally introduced by Eric Winfree and motivated by Wang [21, 22]. Since the available space in this paper is very limited, we just give a very brief introduction and refer to [11, 14] for a more detailed description of DNA-based tile assembly.

We use the notation of [9]. Tiles are mathematical objects that can be implemented with DNA-molecules. A tile may "fill" a cube in the three-dimensional space. Tiles may have one or more *glues* of a certain color attached to each side. Figure 3 explains how the DX-molecule relates to abstract tiles. The sticky ends of the DNA constitute the glues and their colors.

Only glues with the same color may form a stable binding of two tiles. For the sake of modeling, we assume that there is an unlimited supply of every *tiletype* (this is not true for the constituting DNA molecules, but due to the limited size of our desired constructs, it will not be a restriction). Tiles start to randomly form bindings once a *seed tile* is introduced to the medium. The process is illustrated in Figure 4. Figure 4a shows an example set of tiletypes. The self-assembly process starts with a seed-tile in step α_0. In every following step, exactly one tile is added to the assembly. As depicted, the glue strength (the number of black blocks next to the tiles) and the glue labels (the character next to the black blocks) match. The label in the middle can be used for computation.

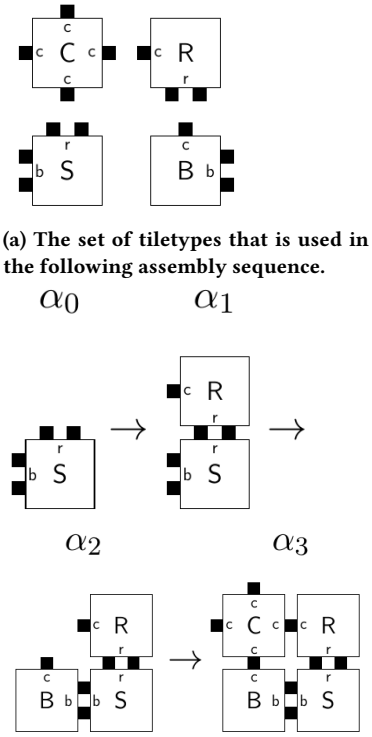


Figure 4: An exemplary assembly sequence of four tiles.

A *Tile Assembly System* (TAS) has a *temperature* τ. The temperature describes how many glues are required to stably attach two tiles to each other. A stable composite of tiles is called an *assembly*. In this paper, we focus on temperature 1 systems.

Most TAS only use two dimensions for simulation. Two-dimensional systems are already capable of universal computation at temperature 2 [4]. Since we are only interested in the geometrical aspect of tile assemblies, we conclude that three-dimensional tiles are of interest and focus on these in all simulations. We already introduced a related approach in [11].

We determined a set of geometrical shapes which are of special interest. Cubes, spheres, ellipsoids, tubes, rectangles and biconcave, flat ellipsoids (erythrocytes) appear useful for in-body application. In the following, we derive algorithms that create sets of tiletypes for the mentioned structures of adjustable sizes.

3 AN ALGORITHM FOR $n \times n \times n$-CUBES

Medical scenarios are among the most prominent areas of application for Nanorobots (e.g. [7, 8, 20]). Experimental research from [2, 13] generated a cube with a controllable lid. It may be used to shelter medical payload from the chemical environment inside the target region. With this level of control, it might be possible to locally apply medication. Through targeted application, the required dosage of medicine may be drastically reduced. The side-effects of medication might also be reduced, since fewer areas of the body are exposed to the substance. In [10], we analyzed numerous medical scenarios.

Since medical payload comes in many shapes, vessels of different, adjustable sizes are of interest. Cubes are especially useful, since they are symmetrical and thus efficient to build.

Cubes have already been assembled in DNA-based tile assembly systems [2, 3, 13]. Most of these structures have been designed manually in a complicated process. We present an algorithm that automatically creates a set of tiletypes for either hollow, framed or filled cubes of adjustable size. In the then following engineering step, at least one tile of each of these tiletypes has to be created. Adding the tiles to a shared medium then leads to the automatic self-assembly of the desired structure. The process is hierarchical. In a first steps the components are created, in a second step they are introduced to a medium and self-assemble into the desired structures.

The basic idea behind the algorithm is the creation of a *distinct* tiletype for every position in the defined cube. This procedure allows for fewer errors, since only one possible tile exists for every position in an assembly, i.e., every tile does only fit in exactly one position. As stated above, the number of possible different tiletypes based on DNA molecules is comparably large, thus this property of the algorithm is no restriction for us, since we are only interested in very small constructs.

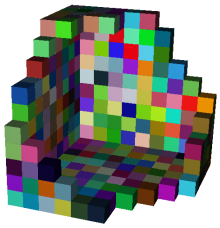
Algorithm 1 displays the computational steps for creating cube tiletypes in pseudocode.

Algorithm 1: Algorithm to create a set of tiletypes that generates a cube of a given edge length.

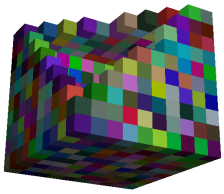
Data: Edge length n, some parameter p
Result: Set of tiletypes T

```
1  x := 0
2  y := 0
3  z := 0
4  T := ∅
5  while x < n do
6      while y < n do
7          while z < n do
8              c := 0
9              for i ∈ {x, y, z} do
10                 if i = 0 ∨ i = n − 1 then
11                     c := c + 1
12             if c ≥ p then
13                 T :=
                       T ∪ {createPositionTiletype(x, y, z, 1)}
14             z := z + 1
15         y := y + 1
16     x := x + 1
```

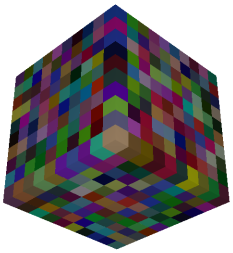
In three nested loops, we consider every possible position of a cube. Lines 9 to 11 store in parameter c whether the current position is an inner position ($c = 0$), a wall position ($c = 1$), an edge position ($c = 2$) or a cornerstone position ($c = 3$). The parameter p then controls which position requires a tile. For a parameter $p = 0$ Algorithm1 creates a solid $n \times n \times n$ cube, since



(a) Start building a $10 \times 10 \times 10$ hollow cube.



(b) The $10 \times 10 \times 10$ cube with a corner still missing.



(c) The fully assembled $10 \times 10 \times 10$ hollow cube.

Figure 5: A hollow cube with an edge length of ten is assembled in the aTAM simulation. The simulation is not interactive so three instances were run with the same algorithm. Different coloring is due to a random assignment of colors to tiletypes when displaying.

every position is filled. Varying the parameter reduces the filling. For $p = 1$ the cube is hollow and for $p = 2$ the cube is just a frame, without even the side walls (only edge and cornerstone tiles).

For every position at which a tile should be placed, we add another tiletype to the output (Line 13). The resulting set of tiletypes may then be simulated in an aTAM simulator which supports three-dimensional tiles.

Figure 5 shows a simulation for the set of tiletypes for a $10 \times 10 \times 10$ hollow cube in three different stages and thus depict the assembly progress. The different (randomly assigned) colors visualize the different tiletypes. The glues themselves were omitted, since they are unique for every position. Every tile has a maximum of one glue on each side.

Cubes are limited structures which do not—or only very rarely—naturally appear inside of the human body. While they are the simplest 3D structure most can imagine, they lack qualities required for efficient transportation inside the human bloodstream. Even the intercellular space is a unique environment with challenges a cube is probably not optimal for. Therefore, we propose different structures which are probably better applicable in the following section.

4 AN ALGORITHM FOR ARBITRARY PREDEFINED THREE-DIMENSIONAL SHAPES

For the creation of arbitrary three-dimensional shapes, we generalized the idea from Algorithm 1. The resulting algorithm can create sets of tiletypes for any given shape. It is also designed for temperature 1 systems. Every tiletype is used exactly once. Since more efficient algorithms exploit symmetries in geometrical figures, a generic (shape-agnostic) algorithm *cannot* be improved to require less tiletypes, in general. This approach always works, but requires the maximum possible amount of tiletypes. We chose temperature 1 systems for the sake of simplicity. They require few additional tiles that are not part of the main assembly. E.g., efficiently assembled squares are based on a binary counter, limiting their growth.

Due to biological limitations with DNA, we consider the number of possible glues in a TAS limited. Once the DNA strands that constitute the glues are too long, they become more prone to binding with wrong counterparts. We therefore assume that only a constant number k of glues and thereby a constant number of tiles is reasonably implementable.

To assemble big shapes nevertheless, we could use efficient sets of tiletypes that can, e.g., assemble squares with less than $O(n)$ tiletypes. Very good algorithms may be able to generate cubes with only $O(n \log(n))$ tiletypes required, or even less [17].

The algorithm for cubes is special, since it offers different variations of the same shape. In general, not every geometrical figure has a "frame" or a "core" that can be displayed separately. We therefore omitted any specific component for the generalized version of the basic general algorithm.

Algorithm 2 takes a finite subset of points from the $\mathbb{R}^3$ as input. The input structure has to be fully connected. Every cube in the structure has to have at least one direct neighbor in the cardinal directions, as well as above or below. The algorithm then outputs a set of tiletypes for the desired shape. The connectedness-constraint is due to limitations in the aTAM definition.

Figure 6 displays an example application of the generalized algorithm on a voxel-based sphere approximation of radius four. The sphere itself is hollow, as Figure 6b shows. Spheres are naturally occurring structures that might be better suited for moving fluid environments than cubes. They equally distribute pressure from the outside along the whole surface of the shape. The other desired shapes—like ellipsoids or rectangles—may also be assembled as voxel-approximations like the sphere.

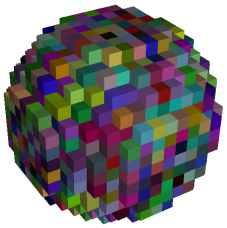
We believe that temperature 1 systems suffice for many practical cases and proofs of concept. The idea is to build "nanoscale" constructs. Very efficiently assembled cubes might very well be of micrometers in size. More efficient algorithms are the subject of future work.

The algorithm which generates the sets of tiletypes is rather efficient due to its simplicity. The simulator which verifies the

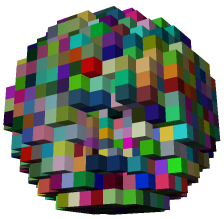
Algorithm 2: Algorithm to create a set of tiletypes which generates a given shape in a Tile Assembly Model.

Data: A boolean array *shape* with three dimensions of max lengths n, m, o
Result: Set of tiletypes T

```
1  x := 0
2  y := 0
3  z := 0
4  T := ∅
5  while x < n do
6      while y < m do
7          while z < o do
8              if shape_{x,y,z} then
9                  T :=
                     T ∪ {createPositionTiletype(x, y, z, 1)}
10             z := z + 1
11         y := y + 1
12     x := x + 1
```



(a) A full hollow sphere produced by the methods for generating arbitrary coherent shapes in the simulator.



(b) The hollow sphere from a different angle shown with a hole to verify the Assembly.

Figure 6: Simulation of the TAS introduced by Algorithm 2. This creates a sphere with radius eight.

output structures on the other hand tends to be slower. Therefore, we analyze the runtime and correctness of the developed software and algorithms.

5 EVALUATION OF SIMULATOR AND ALGORITHMS

The software we developed consists of three major parts, namely (i) a generator implementing algorithms for the synthesis of sets

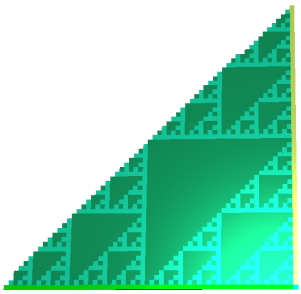


Figure 7: Proof of concept assembly of the Sierpinsky Triangle.

of tiletypes, (ii) the simulation software which verifies the algorithms, and (iii) a component which generates a three-dimensional representation of the assembled structure. We analyze the three parts separately in the following paragraph. We analyze both correctness and runtime.

- The tiletype generator for the 3D aTAM is based on the ISU TAS software [15]. The software implements the mathematical model of Winfree's aTAM rigorously, thus it is working as intended. The previously two-dimensional mathematical objects have been generalized to three dimensions, while keeping the desired behavior intact. To verify the functionality, we tested the software with various sets of tiletypes that have already been evaluated in other papers. Two examples are binary counters and a version of the Sierpinsky Triangle. The sets of tiletypes were modified to fit the 3D context.
  Figure 7 shows the result after several hundred iterations. Only two dimensions were used for the assembly process. The functionality in three dimensions has been verified with algorithms for cubes as displayed in Figure 5.
  The synthesis of the set of tiletypes is dependent on the underlying function used to determine the used positions for the resulting shape. To create each tiletype, only $O(1)$ calculations occur. When using a naive approach to determine the positions that are occupied by tiles, $O(n^3)$ calculations are necessary. The input size n is the biggest size in all of the three dimensions of the given shape. The resulting runtime of the tiletype generation is thus $O(n^3)$.
- The simulation software is slower. In every step of the assembly process, one tile is added to the assembly. Since we are only interested in finite structures, the number of required steps is also denoted by n. The possible positions where a tile may be added are kept in a list. The runtime thus depends on the size of the list of possible positions or *frontier* and the number of steps and generally the number of possible tiles for a position, which is always one in our case. All presented cases are limited by n. The resulting runtime is limited by $O(n^3)$.
- The component for the graphical representation requires $O(n)$ steps.

Since all three steps are independent of each other, the total runtime is limited by $O(n^3)$ and the software works as intended.

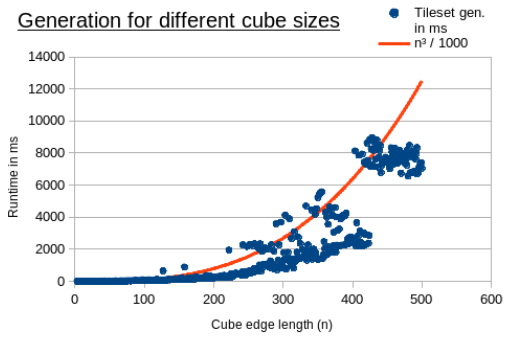


Figure 8: Assembly runtime of cubes of different sizes with respect to the *edge length*.

Figure 8 shows the required time for the synthesis of the set of tiletypes for cubes of various sizes*. The JVM induces overhead due to garbage collection and allocation when the problem instance increases in size. As an example, the run-time values jump between 2.5 seconds and 8 seconds when the cubes edge length grows larger than 400. Additionally, the PC uses a variable CPU clock and is able to increase the calculation frequency if more calculations are required to save energy when running idle.

6 CONCLUSION

The algorithms and the simulator itself have been tested. Both represent first sound steps in the process of automatically designing architectures for nanostructures or more specific nanoscale machines/robots. Other algorithms for better geometrical objects will be introduced in the future and tested in both the aTAM simulator and a more realistic version (the kTAM [22]) which is still in development.

Algorithm 2 is a generalized procedure to generate any connected shape in form of a set of tiletypes. Algorithm 1 exploits symmetries in shapes like squares or cubes to generate variations of the basic structure. Algorithms for specific shapes may be more efficient than general versions.

The simulation software implements Winfree's aTAM model and is capable of verifying the assembly process of sets of tiletypes. Unlike Xgrow of ISU-TAS [**?**], the implemented software is capable of simulating growth in three dimensions.

Possible future research focuses on the following aspects:

- Generating efficient algorithms for more complex structures, like red blood cell-like shapes or simple spheres/cubes.
- Implementing another simulation tool capable of both respecting errors with DNA implementations and using three-dimensional tiles.
- Implementing error-correction measures for arbitrary shapes in the kTAM model.
- Developing a modular principle to generate complex architectures for Nanostructures from basic components.
- Defining assembly sequnces that prohibit growth of "outer" structures before the "inner" structures are finished.

The often complicated process of designing the tilesets of three-dimensional shapes of specific sizes may now be partly automated. The user simply has to supply a binary description of the target shape and receives the desired output. Only the design of the actual DNA implementation is still necessary.

*The JVM used was OpenJDK version 1.8.0_144 64-Bit Server VM (build 25.144-b01, mixed mode).

REFERENCES

[1] Ian F. Akyildiz, Fernando Brunetti, and Cristina Blázquez. 2008. Nanonetworks: A new communication paradigm. *Computer Networks* 52, 12 (2008), 2260–2279.

[2] Ebbe S. Andersen, Mingdong Dong, Morten M. Nielsen, Kasper Jahn, Ramesh Subramani, Wael Mamdouh, Monika M. Golas, Bjoern Sander, Holger Stark, Cristiano L. P. Oliveira, Jan Skov Pedersen, Victoria Birkedal, Flemming Besenbacher, Kurt V. Gothelf, and Jorgen Kjems. 2009. Self-assembly of a nanoscale DNA box with a controllable lid. 459, 7243 (2009), 73–76. https://doi.org/10.1038/nature07971

[3] Florent Becker, Éric Rémila, and Nicolas Schabanel. [n. d.]. Time Optimal Self-assembly for 2D and 3D Shapes: The Case of Squares and Cubes. In *DNA Computing: 14th International Meeting on DNA Computing, DNA 14, Prague, Czech Republic, June 2-9, 2008. Revised Selected Papers*, Ashish Goel, Friedrich C. Simmel, and Petr Sosík (Eds.). Springer Berlin Heidelberg, 144–155. http://dx.doi.org/10.1007/978-3-642-03076-5_12 DOI: 10.1007/978-3-642-03076-5_12.

[4] R. Berger. 1966. *The Undecidability of the Domino Problem.* American Mathematical Society. https://books.google.ie/books?id=8AmiHD0Lbu8C

[5] Florian Büther, Florian-Lennert Lau, Marc Stelzner, and Sebastian Ebers. [n. d.]. A Formal Definition for Nanorobots and Nanonetworks. In *Internet of Things, Smart Spaces, and Next Generation Networks and Systems: 17th International Conference, NEW2AN 2017, 10th Conference, ruSMART 2017, Third Workshop NsCC 2017, St. Petersburg, Russia, August 28–30, 2017, Proceedings*, Olga Galinina, Sergey Andreev, Sergey Balandin, and Yevgeni Koucheryavy (Eds.). Springer International Publishing, 214–226. https://doi.org/10.1007/978-3-319-67380-6_20 DOI: 10.1007/978-3-319-67380-6_20.

[6] Mao Chengde. 2004. Basic DNA Structures for Self-Assembly (C) a DNA DX. (2004). https://en.wikipedia.org/wiki/File:Mao-DX-schematic-2.svg License: https://creativecommons.org/licenses/by/2.5/deed.en.

[7] Diogo Caldeira Ferreira, Luis Paulo Reis, and Nuno Vasco Lopes. 2017. A nanocommunication system for endocrine diseases. *Cluster Computing* (2017), 1–18.

[8] Robert A. Freitas. 2005. Current Status of Nanomedicine and Medical Nanorobotics. *Journal of Computational and Theoretical Nanoscience* 2, 1 (2005), 1–25.

[9] James I. Lathrop, Jack H. Lutz, and Scott M. Summers. [n. d.]. Strict Self-assembly of Discrete Sierpinski Triangles. In *Computation and Logic in the Real World: Third Conference on Computability in Europe, CiE 2007, Siena, Italy, June 18-23, 2007. Proceedings*, S. Barry Cooper, Benedikt Löwe, and Andrea Sorbi (Eds.). Springer Berlin Heidelberg, 455–464. http://dx.doi.org/10.1007/978-3-540-73001-9_47 DOI: 10.1007/978-3-540-73001-9_47.

[10] Florian Lau, Florian Büther, and Bennet Gerlach. 2017. Computational Requirements for Nano-Machines: There is Limited Space at the Bottom. In *4th ACM International Conference on Nanoscale Computing and Communication 2017*. ACM, Washington DC, USA.

[11] Florian Lau and Stefan Fischer. 2017. Embedding Space-Constrained Quantum-Dot Cellular Automata in Three-Dimensional Tile-Based Self-Assembly Systems. In *4th ACM International Conference on Nanoscale Computing and Communication 2017 (ACM NanoCom'17)*. ACM, Washington DC, USA.

[12] S Li, Q Jiang, S Liu, Y Zhang, Y Tian, C Song, J Wang, Y Zou, GJ Anderson, JY Han, Y Chang, Y Liu, C Zhang, L Chen, G Zhou, G Nie, H Yan, B Ding, and Y Zhao. 2018-02-12 00:00:00.0. A DNA nanorobot functions as a cancer therapeutic in response to a molecular trigger in vivo. *Nature Biotechnology* 5 (2018-02-12 00:00:00.0). https://doi.org/10.1038/nbt.4071

[13] Kao Ming-Yang and Vijay Ramachandran. [n. d.]. DNA Self-Assembly For Constructing 3D Boxes. In *Algorithms and Computation: 12th International Symposium, ISAAC 2001 Christchurch, New Zealand, December 19–21, 2001 Proceedings*, Peter Eades and Tadao Takaoka (Eds.). Springer Berlin Heidelberg, 429–441. https://doi.org/10.1007/3-540-45678-3_37 DOI: 10.1007/3-540-45678-3_37.

[14] Matthew J. Patitz. [n. d.]. An Introduction to Tile-Based Self-assembly. In *Unconventional Computation and Natural Computation: 11th International Conference, UCNC 2012, Orléan, France, September 3-7, 2012. Proceedings*, Jérôme Durand-Lose and Nataša Jonoska (Eds.). Springer Berlin Heidelberg, 34–62. http://dx.doi.org/10.1007/978-3-642-32894-7_6 DOI: 10.1007/978-3-642-32894-7_6.

[15] Matthew J. Patitz. 2011. Simulation of Self-Assembly in the Abstract Tile Assembly Model with ISU TAS. *CoRR* abs/1101.5151 (2011). arXiv:1101.5151 http://arxiv.org/abs/1101.5151

[16] Paul W. K. Rothemund. 2006. Folding DNA to create nanoscale shapes and patterns. *Nature* 440, 7082 (16 Mar 2006), 297–302. https://doi.org/10.1038/nature04586

[17] Paul W. K. Rothemund and Erik Winfree. [n. d.]. The Program-size Complexity of Self-assembled Squares (Extended Abstract). In *Proceedings of the Thirty-second Annual ACM Symposium on Theory of Computing* (2000) *(STOC '00)*. ACM, 459–468. https://doi.org/10.1145/335305.335358

[18] Nadrian C. Seeman. 1982. Nucleic acid junctions and lattices. *Journal of Theoretical Biology* 99, 2 (1982), 237 – 247. https://doi.org/10.1016/0022-5193(82)90002-9

[19] Nadrian C. Seeman. 2005. Structural DNA Nanotechnology: An Overview. 303 (2005), 143–166. https://doi.org/10.1385/1-59259-901-X:143

[20] Marc Stelzner, Florian-Lennert Lau, Katja Freundt, Florian Büther, Mai Linh Nguyen, Cordula Stamme, and Sebastian Ebers. 2016. Precise Detection and Treatment of Human Diseases Based on Nano Networking. In *11th International Conference on Body Area Networks*. EAI, Turin, Italy.

[21] Hao Wang. [n. d.]. Dominoes and the Aea Case of the Decision Problem. In *Computation, Logic, Philosophy: A Collection of Essays*. Springer Netherlands, 218–245. https://doi.org/10.1007/978-94-009-2356-0_11 DOI: 10.1007/978-94-009-2356-0_11.

[22] Erik Winfree. [n. d.]. Simulations of Computing by Self-Assembly. In *University of Pennsylvania* (1998-05-31). 16–19.